

TECHNOLOGICAL UNIVERSITY DUBLIN

CITY CAMPUS - GRANGEGORMAN

TU856 BSc. (Honours) Degree in Computer Science

TU858 BSc. (Honours) Degree in Computer Science
(International)

Year 3

SEMESTER 1 EXAMINATIONS 2024/25

CMPU3006, Client Server Programming

Internal Examiner: Mr. Damian Bourke

Dr. Paul Doyle

External Examiner: Dr. Colm O'Riordan

Exam Duration: 2 hours.

Attempt Question 1 and three of the four remaining questions. All questions carry equal marks.

1. Figure 1 (A snippet of *daytime* client code) reveals some code for a *daytime* client with error checking code removed for brevity. Refer to this figure and to the following list of terms:

servAddr, inet_ntop, > 0 (the number zero), sockaddr, clntAddr, sin_port, sock, htons, inet_pton, == 0 (the number zero), AF_INET, atoi, sin_port, clntSock, sockaddr_in.

```

43 .....IGNORE ALL PREVIOUS LINES OF CODE.....
44
45     if (connect(sock, (struct XXXXX *) &servAddr, sizeof(servAddr)) < 0)
46
47     int clntSock = accept(sock, (struct sockaddr *) NULL, NULL);
48
49     snprintf(sendbuffer, sizeof(sendbuffer), "%.24s\r\n", ctime(&ticks));
50
51     ssize_t numBytesSent = send(clntSock, sendbuffer, strlen(sendbuffer), 0);
52
53     close(XXXXX);
54
55     while ((numBytes = recv(XXXXX, recvbuffer, BUFSIZE - 1, 0)) XXXXX) {
56         recvbuffer[numBytes] = '\0';
57         fputs(recvbuffer, stdout);
58     }
59
60     fputc('\n', stdout);
61
62     exit(0);
63 }

```

Figure 1: A snippet of *daytime* client code.

- (a) There are some errors introduced into this code relating to inappropriate and misplaced primitive calls. Using the line numbers provided, identify which lines of code should be removed and the correct location for any misplaced code. Give reasons for your selections. (8 marks)
- (b) Using the line numbers provided, identify which XXXXX value can be replaced with which term from the above list. (10 marks)
- (c) Refer to the *while()* loop commencing on line 55. State if it is necessary, for the correct implementation of the *daytime* application, to add any additional code within the loop structure to ensure an exit from this loop. If you think it is necessary, identify the code and if not, explain why. (7 marks)

2. Figure 2 (*A HTTP/1.1 Header*) represents an **incorrectly** formatted Header section from a **HTTP/1.1** Response message sent from the TU Dublin webserver – the Body section is deliberately omitted:

Response Message

1. HTTP/1 OK 200\r\n
2. Date: Sat, 02 Nov 2024 12:03:38 GMT\r\n
3. Content-Type: text/html; charset=UTF-8\r\n
4. Connection: close\r\n\r\n
5. Content-Length: 232\r\n
6. Server Apache\r\n
7. Cache-Control: max-age=172800\r\n
8. Expires: Mon, 04 Nov 2024 12:03:38 GMT\r\n
9. \r\n

Figure 2: A HTTP/1.1 Header.

- (a) There are several **syntax** errors deliberately included in this response header - the errors are spread across three lines. Using the line numbers provided, re-write each incorrect line using correct syntax – exclude lines that are correct. (9 marks)
- (b) The entire HTTP Response message, comprising Header and Body sections, contains 528 bytes. Assuming the characters \r and \n are each counted as a single byte, calculate the number of bytes contained in the Header Section. Justify your answer with reference to any data contained in Header section. (6 marks)
- (c) The client application uses a *recv-while()* loop structure to read the entire response message i.e. the Header and Body sections, into an application layer buffer. This coding structure relies on testing the return value of the call to *recv()* to determine when to break from the loop. Outline a coding strategy to determine when the client application should terminate this loop without relying on the special case when the return value is zero. (10 marks)

3. In relation to the *listen()* primitive and the associated TCP *connections* queue:

- (a) One of the actions performed by the call to the *listen()* primitive is to transform an *active* socket into a *passive* socket. Outline the key difference between each type of socket and identify which application, client or server, each is associated with.

(6 marks)

- (b) Describe the operation of the TCP *connections* queue. In your answer, refer to the TCP messages involved in a connection request explaining how TCP determines the placement of entries into each of the *incomplete* and *complete* queues and, explain how TCP treats connection requests arriving when the queue is full.

(12 marks)

- (c) Identify the primitive within a server application that impacts on the connections queue. In your answer, explain how TCP deals with data arriving on connections within the queue.

(7 marks)

4. Refer to Figure 3 (*Screenshot from Wireshark*). This screenshot shows a sequence of segment exchanges between host machines hosting a *http* client and *http* server application.

	Source	Destination	Protocol	Info
1	93.184.215.14	10.0.2.15	TCP	80 → 51294 [ACK] Seq=1 Ack=71 Win=6553
2	93.184.215.14	10.0.2.15	HTTP	HTTP/1.1 200 OK (text/html)
3	10.0.2.15	93.184.215.14	TCP	51294 → 80 [ACK] Seq=71 Ack=1618 Win=3
4	93.184.215.14	10.0.2.15	TCP	80 → 51294 [FIN, ACK] Seq=1618 Ack=71
5	10.0.2.15	93.184.215.14	TCP	51294 → 80 [ACK] Seq=71 Ack=1619 Win=3

Figure 3: Screenshot from Wireshark.

- (a) Separately identify the *Socket Pair* for the TCP connection from the perspective of each host.

(4 marks)

- (b) With reference to the *Three Phases of Communication*, identify the phase under commencement in line 4. Justify your answer.

(3 marks)

(Question continues on next page)

- (c) ACKs are used to acknowledge data received. With reference to the ACKs contained in the segments shown in lines 4 & 5, separately identify which *Phase(s) of Communication*, each ACK relates to. Justify your answer.

(6 marks)

- (d) In relation to the *Phase of Communication* under commencement in line 4, identify any additional messages that will need to be exchanged to complete this phase. In your answer, identify the host from which each of these messages should originate and, the values in the associated SEQ and ACK fields.

(12 marks)

5. Refer to Figure 4 (*Screenshot from Wireshark*). This screenshot shows a sequence of segment exchanges between hosts hosting a *http* client and *http* server application.

	Source	Destination	Protocol	Info
1	10.0.2.15	93.184.215.14	TCP	51294 → 80 [SYN] Seq=0
2	93.184.215.14	10.0.2.15	TCP	80 → 51294 [SYN, ACK] Seq=0 Ack=1
3	10.0.2.15	93.184.215.14	TCP	51294 → 80 [ACK] Seq=1 Ack=1
4	10.0.2.15	93.184.215.14	HTTP	GET /index.html HTTP/1.1
5	93.184.215.14	10.0.2.15	TCP	80 → 51294 [ACK] Seq=1 Ack=71
6	93.184.215.14	10.0.2.15	HTTP	HTTP Response (text/html)
7	10.0.2.15	93.184.215.14	TCP	51294 → 80 [ACK] Seq=71 Ack=1618

Figure 4: Screenshot from Wireshark.

- (a) With reference to line 4, identify the starting and ending byte numbers for the data contained in the segment's payload field. Justify your answer with reference to data contained elsewhere in the screenshot.

(6 marks)

- (b) With reference to line 6, identify the starting and ending byte numbers for the data contained in the segment's payload field. Justify your answer with reference to data contained elsewhere in the screenshot.

(6 marks)

- (c) Line 7 shows an ACK message leaving host 10.0.2.15 in response to data received from host 93.184.215.14. For TCP, ACK messages for data received must be returned within a specific time-period, as monitored by an RTO timer. Explain how either or both hosts would respond if this timer expired. In your answer, identify which host(s) sets and monitors the RTO timer.

(5 marks)

- (d) There are two factors that influence the value for the RTO timer – *RTT* and *variance*. Explain these factors and outline how they are used in the calculation of the maximum value for the RTO timer.

(8 marks)